Gresh.

Keycloak in Practice

Theme customization, email templates, SPI extensions and the single-node & cluster deployments

Personal reference guide · Todimu

Keycloak 26.7.3 · project `keycloak-setup`

October 2026

Contents

1. **The project at a glance**: repo layout and how the pieces fit
2. **Login theme**: inheritance, CSS-only design, logo, wording, gotchas
3. **Email theme**: one wrapper template, messages, greeting, logo, event emails
4. **Extending Keycloak with SPIs**: the pattern, config, packaging, SPI catalogue
5. **Case study: Kafka event listener**: design, configuration, lessons learned, operations
6. **Deployments**: single node on AWS EC2 and the 3-node cluster
7. **Command cheat sheet**
8. **Appendix**: full source of the files built in this project

> **How to read this guide.** Everything here was built and tested on Keycloak 26.7.3. Where something is a recommendation rather than what was actually built, it is labelled *Improvement*. The deployment chapter combines the `keycloak-setup` repository with notes from earlier conversations about the AWS deployment; those parts are marked as such.

1. The project at a glance

One Maven multi-module project produces two jars: a **theme** (files only) and a **Java extension**. A multi-stage Dockerfile builds both and bakes them into a custom Keycloak image, which Docker Compose runs as a three-node cluster behind nginx.

```
keycloak-setup/
├── pom.xml                        parent (packaging=pom), modules: keycloak-theme, keycloak-
cutomization
├── Dockerfile                     maven build → kc.sh build (providers baked in) → runtime image
├── docker-compose.yaml            nginx-lb, iam-1..3, MySQL, MailHog
├── nginx-lb/default.conf          least_conn load balancer over iam-1..3
├── .env                           KC_* settings and secrets (git- and docker-ignored)
├── .dockerignore                  .env, .git, .idea, **/target, ...
├── keycloak-theme/                theme jar (no Java code)
│   └── src/main/resources/
│       ├── META-INF/keycloak-themes.json
│       └── theme/greshfi/
│           ├── login/   theme.properties, messages/, resources/css/greshfi.css, resources/img/
│           └── email/   theme.properties, html/template.ftl, messages/
└── keycloak-cutomization/         Java extension jar (Kafka event listener)
    └── src/main/
        ├── java/com/todimu/keycloak/events/   Factory, Provider, Publisher
        └── resources/META-INF/services/org.keycloak.events.EventListenerProviderFactory
```

Piece	What it is	Where it ends up
Theme jar	Templates, CSS, images, message files plus `keycloak-themes.json`	`/opt/keycloak/providers/`
Extension jar	Java SPI implementation + shaded `kafka-clients`	`/opt/keycloak/providers/`
`kc.sh build`	Bakes providers and build-time options into an optimized server	Image layer; runtime starts with `start --optimized`
Realm settings	Which theme/listener each realm uses	Database (admin console)

> **Rule of thumb:** files and Java code live in the image (rebuild to change them); per-realm choices live in the admin console (no rebuild).

2. Login theme

2.1 Core idea: inherit, don't copy

Keycloak themes form a parent chain. A page is looked up in your theme first, then its parent, then the parent's parent. So your theme only needs the files you actually change; everything else is inherited and keeps receiving upstream fixes when Keycloak is upgraded.

```
greshfi/login  ──parent──▶  keycloak.v2/login  ──parent──▶  base/login
(your CSS, messages)       (PatternFly 5 templates,       (all 46 page templates,
                            styles.css, dark mode)         the original logic)
```

Parent	Look	Notes
`keycloak.v2`	PatternFly 5, modern, dark mode	Default login theme in Keycloak 26; **used for greshfi/login**
`keycloak`	PatternFly 4 (older look)	Templates come from `base`
`base`	Unstyled HTML	Marked `abstract=true`; meant to be inherited, not selected

2.2 Minimal structure

`META-INF/keycloak-themes.json`

```
{
  "themes": [{
    "name" : "greshfi",
    "types": [ "login", "email" ]
  }]
}
```

`theme/greshfi/login/theme.properties`

```
locales=…                          (kept from the original copy)
parent=keycloak.v2
styles=css/styles.css css/greshfi.css
```

> `styles=` **replaces** the parent's list, so always keep the parent's `css/styles.css` first and add yours after it. Files that are not in your theme (like `css/styles.css`) are found in the parent.

2.3 What lives where

Want to change…	Edit	Notes
Colours, spacing, shapes, background	`resources/css/greshfi.css`	90% of a redesign. Uses PatternFly 5 classes (`pf-v5-c-…`)
Wording	`messages/messages_en.properties`	e.g. `loginAccountTitle`, `usernameOrEmail`, `doLogIn`
Logo / images	`resources/img/` + CSS	Referenced from CSS as `../img/…`

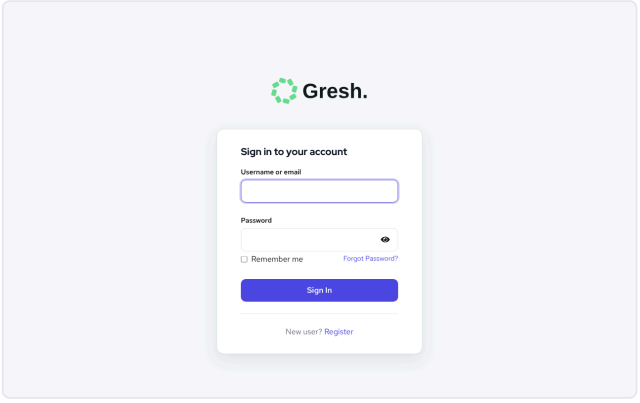
Want to change…	Edit	Notes
Browser tab icon	`resources/img/favicon.ico`	Picked up automatically, no code
Page structure (layout, new elements)	Copy **one** `.ftl` from `keycloak.v2`	Only when CSS cannot do it; most themes own just `template.ftl`
Name above the card	Realm settings → General → (HTML) Display name	Not a theme file

When you do own an .ftl, never break these

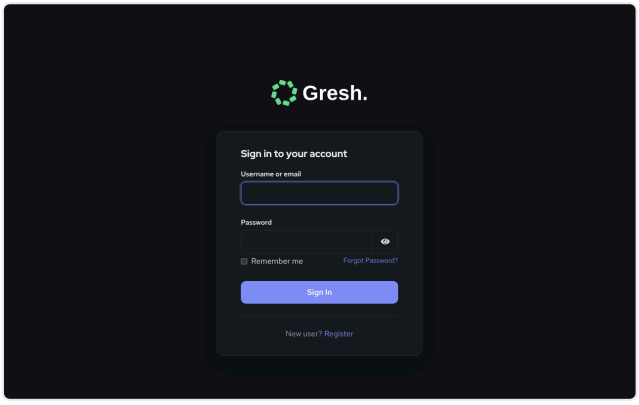
- the form's `action="${url.loginAction}"` and `method="post"`, the input `name`s and hidden inputs
- the `<#nested "…">` slots in `template.ftl` (each page fills them)
- the message / `messagesPerField` error blocks, and `${msg("…")}` for text (translations)
- FreeMarker escaping: never add `?no_esc` to user-controlled values

2.4 The CSS-only design (greshfi)

The whole redesign is one stylesheet layered on top of `keycloak.v2`: brand colours defined once as CSS variables (and swapped under `.pf-v5-theme-dark`), a rounded card, unified rounded inputs (PatternFly's `::before/::after` borders turned off), a full-width primary button, and error states that stay red even when focused. Because it targets the standard classes, every inherited page (register, reset password, OTP, errors…) picks it up automatically. Full file in the appendix.



Light mode



Dark mode (follows the OS)

Element	Selector
Page background	`.login-pf body`
Header / realm name	`#kc-header-wrapper`
Card	`.pf-v5-c-login__main`, header `.pf-v5-c-login__main-header`
Inputs	`.pf-v5-c-form-control`, password + eye: `.pf-v5-c-input-group`
Primary button	`.pf-v5-c-button.pf-m-primary`, sign-in button `#kc-login`
Dark mode hook	`.pf-v5-theme-dark` (added by keycloak.v2 when the OS is dark; `darkMode=false` disables)

Element	Selector
Field in error	`input[aria-invalid="true"]`

2.5 Logo: icon and wordmark in one image

1. Put the SVG in `login/resources/img/` (light and dark variants).
2. Set the realm's **HTML Display name** to `<div class="kc-logo-text"><span>Greshfi</span></div>`.
3. Point that element at your image:

```
div.kc-logo-text {
  background-image: url("../img/logo-dark-text.svg");
  width: 200px;
  height: 58px;
}
.pf-v5-theme-dark div.kc-logo-text {
  background-image: url("../img/logo-light-text.svg");
}
```

Three realm fields that look alike

Field	Effect
Realm name	The ID in URLs (`/realms/<name>/…`). Renaming changes every issuer and redirect URL.
Display name	Plain text, used only when HTML Display name is empty.
HTML Display name	What the login header actually renders. On `master` it defaults to the Keycloak logo markup.

2.6 Gotchas (all hit during this project)

Symptom	Cause / fix
Theme missing from the dropdown	`abstract=true` in `theme.properties`, or the theme reuses a built-in name (`base`, `keycloak`, `keycloak.v2`)
Keycloak logo still showing	Realm's HTML Display name still contains `kc-logo-text` markup with the default image
CSS change not visible after rebuild	Browser cache (resources are cached up to 30 days and the URL doesn't change): use incognito / DevTools 'Disable cache'. Or stale image tag, or not all load-balanced nodes recreated
Mixed old/new look	Leftover `.ftl` copies from a different parent overriding the real parent's templates
Copied 46 templates 'just in case'	They silently pin you to that version; on upgrade, fixes and new flows skip you. Delete unchanged copies

Which login pages exist (all render inside template.ftl)

Sign-in (`login.ftl`, username/password split, OTP, passkeys, select authenticator), registration (`register.ftl`, terms), credential flows (reset/update password, configure OTP, recovery codes, WebAuthn), required actions (verify email, update profile/email), identity-provider linking, OAuth consent and device code, logout pages, and generic `info.ftl` / `error.ftl` / `login-page-expired.ftl`.

2.7 Fast iteration

```
# dev container with the theme folder mounted and caching off: edit → refresh
command: start-dev --spi-theme--cache-themes=false --spi-theme--cache-templates=false
volumes:
  - ./keycloak-theme/src/main/resources/theme/greshfi:/opt/keycloak/themes/greshfi
```

> Different theme per realm: Realm settings → Themes. Per application: Clients → *client* → Login settings → Login theme. One jar can contain several themes; a child theme (`parent=greshfi`) only overrides what differs. For theme choice by domain/organization there is a **ThemeSelectorProvider** SPI (only one can be active).

3. Email theme

3.1 How an email is assembled

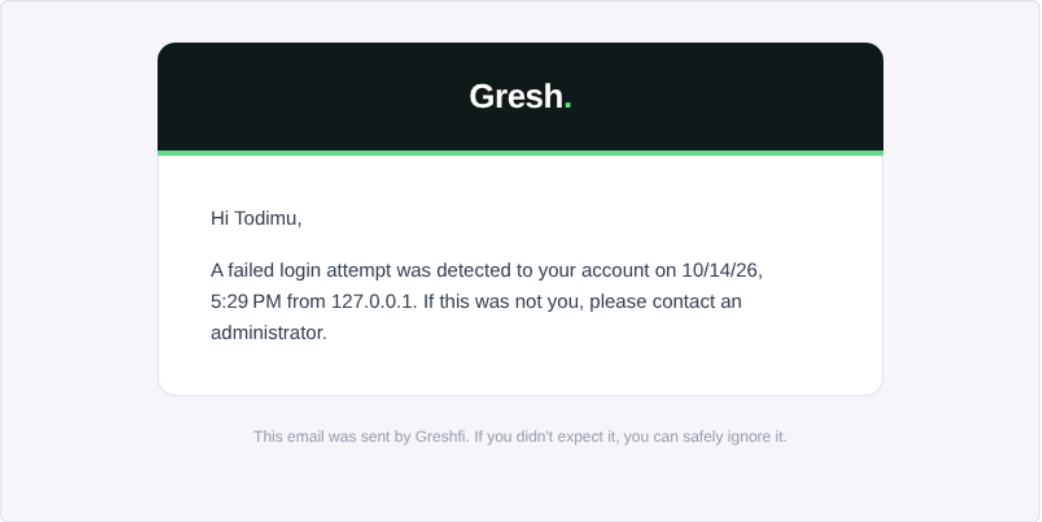
```
1. Keycloak (Java) decides to send e.g. LOGIN_ERROR and collects data: user, event, realmName, link, …
2. The per-email template (inherited from base), e.g. event-login_error.ftl:
       <@layout.emailLayout>
         ${msg("eventLoginErrorBodyHtml", event.date, event.ipAddress)}
       </@layout.emailLayout>
3. msg() fills {0}, {1} … in messages_en.properties
4. The finished body is dropped into YOUR wrapper html/template.ftl at <#nested>
       header (logo) + "Hi Todimu," + [body] + footer
```

Layer	File	Job
Data	Keycloak Java code	Decides which email, supplies values
Per-email template	`password-reset.ftl` , `event-login_error.ftl` … (inherited)	Picks the message key, passes values
Wording	`messages_en.properties`	Subject + body with `{0}` placeholders
Wrapper	**your** `html/template.ftl`	Same frame for every HTML email

3.2 Setup that was built

```
greshfi/email/
├── theme.properties     parent=keycloak  (templates come from base; keycloak.v2 has no email type)
├── html/template.ftl    the only template owned: branded wrapper
└── messages/            subjects and wording
(text/ folder deleted: plain-text versions are inherited)
```

- **No CSS files**: email clients ignore external stylesheets, so styles are inline (plus a small `<style>` block for links/paragraphs coming from messages). Tables for layout.
- **Two versions per email**: `html/` and `text/` (plain-text fallback, sent together). Each email has two message keys, e.g. `passwordResetBodyHtml` and `passwordResetBody`; keep them in sync. Subjects are shared.
- **Greeting**: the wrapper adds `${msg("emailGreeting", user.firstName?cap_first)}` when a first name exists (`emailGreeting=Hi {0},`).
- **Styling inside messages is stripped**: `kcSanitize` removes `style=` attributes, so style in `template.ftl` instead.
- **Logo**: emails need a public https URL to a **PNG** (Gmail/Outlook don't render SVG). Set `logoUrl=` in `email/theme.properties`; without it a text wordmark is shown. Keycloak can host it from the login theme at `/resources/<version-code>/login/greshfi/img/…` , but that code changes on Keycloak upgrade.



A real failed-login alert rendered through the wrapper

3.3 Sending and testing

Item	Where
SMTP server, From, Reply-To	Realm settings → Email (MailHog locally: host `mailhog`, port `1025`, UI `localhost:8025`)
Use the theme	Realm settings → Themes → Email theme
Verify email / Forgot password	Realm settings → Login toggles
Security alerts (failed login, password changed…)	Add the built-in `email` event listener (Realm settings → Events). Only users with a *verified* email; unknown users get nothing (no account enumeration)
Admin-triggered emails	Admin API: `executeActionsEmail(List.of("VERIFY_EMAIL","UPDATE_PASSWORD"))` (one email, several actions) vs `sendVerifyEmail()` (verification only)
Production deliverability	SPF, DKIM, DMARC on the sending domain; test in real Gmail and Outlook

> The failed-login alert fires on **every** failed attempt. Pair it with brute-force protection (Realm settings → Security defenses) or a custom listener with rate limiting.

4. Extending Keycloak with SPIs

4.1 The pattern (same for every extension type)

Piece	Lifecycle	Responsibility
Factory (… `ProviderFactory`)	One instance, created at startup	`getId()` , read config in `init(Config.Scope)` , own shared resources (clients, thread pools), `create(session)` , `close()`
Provider	One per request/session	The actual behaviour; must be cheap; never owns shared resources
Service file	Read at build/startup	`META-INF/services/<factory interface FQN>` containing your factory's class name (Java ServiceLoader). A typo means Keycloak silently ignores it
Packaging	Build time	Jar in `/opt/keycloak/providers/` , then `kc.sh build`

4.2 Configuration: Config.Scope

```
KC_SPI_EVENTS_LISTENER__KAFKA__BOOTSTRAP_SERVERS=kafka:9092
       └──── SPI ────┘ └ id ┘ └─ property ────┘     id must equal factory.getId()

CLI:             --spi-events-listener--kafka--bootstrap-servers=kafka:9092
keycloak.conf:   spi-events-listener--kafka--bootstrap-servers=kafka:9092

@Override public void init(Config.Scope config) {
    String servers = config.get("bootstrap-servers", "kafka:9092");   // kebab and camelCase both work
(verified)
    int queue      = config.getInt("queue-size", 10000);
    String[] types = config.getArray("event-types");                  // comma-separated → array
}
```

- Read config only in the factory's `init()` , pass values to providers via the constructor.
- Always give defaults; validate and fail loudly only for genuine misconfiguration.
- Prefer `Config.Scope` over `System.getenv` so CLI and `keycloak.conf` keep working.

4.3 Packaging rules (pom.xml)

Dependency	Scope	Why
`keycloak-server-spi` , `-spi-private` , `keycloak-core` , `jboss-logging`	`provided`	Keycloak already has them; bundling them causes startup/linkage errors
`kafka-clients` (any library Keycloak lacks)	compile	Must ship inside your jar → `maven-shade-plugin`
`org.slf4j:*`	excluded from the shaded jar	Keycloak ships its own copy

> `ServicesResourceTransformer` in the shade plugin merges `META-INF/services` files instead of letting one overwrite another. Relocating Kafka packages was dropped: Keycloak has no Kafka to clash

> with, and Kafka loads classes by name.

4.4 SPI catalogue

Extension	Factory interface (= services file name)	Turned on via
Event listener	`org.keycloak.events.EventListenerProviderFactory`	Realm settings → Events → Event listeners
Authenticator (login step)	`org.keycloak.authentication.AuthenticatorFactory`	A step in an authentication flow
Required action	`org.keycloak.authentication.RequiredActionFactory`	Authentication → Required actions
Protocol mapper (token claims)	`org.keycloak.protocol.ProtocolMapper`	Client / client scope mappers
User storage (external DB)	`org.keycloak.storage.UserStorageProviderFactory`	User federation
Custom REST endpoints	`org.keycloak.services.resource.RealmResourceProviderFactory`	Live at `/realms/<realm>/<id>`
Identity provider (e.g. Apple)	`org.keycloak.broker.social.SocialIdentityProviderFactory`	Identity providers
Email sender	`org.keycloak.email.EmailSenderProviderFactory`	Server option
Theme selector	`org.keycloak.theme.ThemeSelectorProviderFactory`	`--spi-theme-selector--provider=<id>` (one active)

Some SPIs are *internal* (event listener, email sender): Keycloak logs `KC-SERVICES0047 … internal SPI` at startup. That is expected; re-test after upgrades. Themes are the exception to the Java pattern: they are files registered via `keycloak-themes.json`.

Next extension when needed: Authenticator

Same pattern (`Authenticator` + `AuthenticatorFactory`). New concepts: `authenticate()` / `action()` , driving the flow with `context.success()` , `context.failure()` , `context.challenge(form)` , adding it to a flow, and an optional `.ftl` in the login theme for its screen. Start from the official keycloak-quickstarts `extension/` examples.

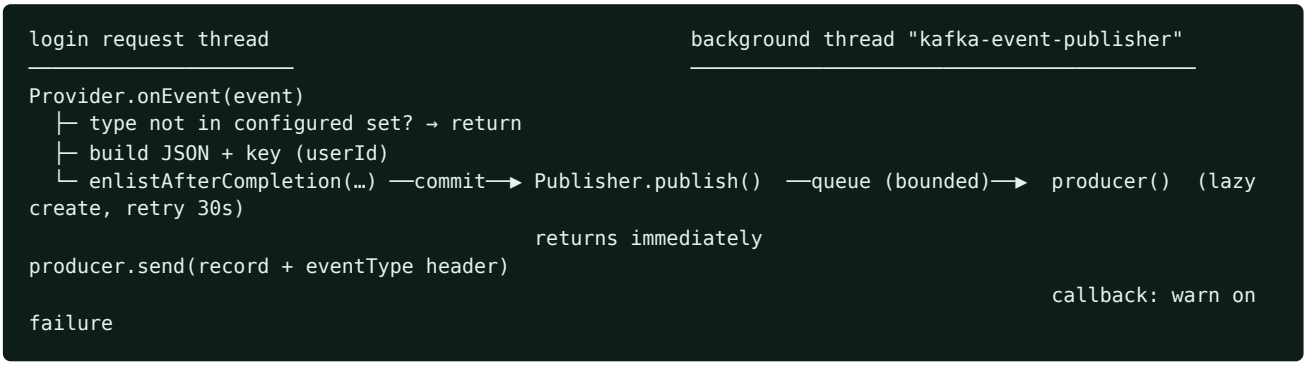
5. Case study: Kafka event listener

Publishes a configurable list of Keycloak user events to a Kafka topic, so other systems (fraud monitoring, analytics, audit) can react to logins, failures and registrations.

5.1 What Keycloak sends

- **User events**: `onEvent(Event)` is called for **every** user event in realms where the listener is enabled. 132 `EventType` values in 26.7.3, mostly pairs like `LOGIN` / `LOGIN_ERROR`. Filtering is your own code.
- **Admin events**: `onEvent(AdminEvent, …)`, operation (CREATE/UPDATE/DELETE/ACTION) × resource type (USER, CLIENT…). Not published by this listener (yet).
- Each event is delivered once, on the node that handled the request. 'Saved types' in the realm only affect Keycloak's DB storage, not listeners.
- `LOGIN_ERROR` carries `error`: `invalid_user_credentials`, `user_not_found`, `user_disabled`, `user_temporarily_disabled`, `invalid_code` …

5.2 Design



Decision	Why
Filter first in `onEvent`	It runs inside the user's request; `REFRESH_TOKEN` alone would flood the topic
Publish after commit (`enlistAfterCompletion`)	Never announce something that was rolled back
Single background thread + bounded queue	`producer.send()` blocks up to `max.block.ms` when Kafka is down. Measured: +2 s on every login before the change, ~50–70 ms after. Full queue → drop with warning, no unbounded memory
Producer created lazily on the background thread, retried every 30 s	An unresolvable broker host made `new KafkaProducer` throw at startup and took all 3 nodes down. Now Keycloak always starts and reconnects on its own
Context class loader = Kafka's loader while creating the producer	Kafka loads classes by name (serializers, SASL login modules via JAAS). Under Keycloak/Quarkus the thread's loader can't see the jar; clearing it to `null` broke JAAS
Key = userId (fallback realmId)	All events of one user land on one partition, in order
`acks=all`, idempotence on	No duplicates or silent loss on retries

Decision	Why
Explicit JSON payload	Consumers don't depend on Keycloak's internal `Event` class
Fail fast only on bad config	Username without password or unknown SASL mechanism stops startup; Kafka being unavailable does not

5.3 Configuration

Env var (`KC_SPI_EVENTS_LISTENER__KAFKA__…`)	Default	Meaning
`BOOTSTRAP_SERVERS`	`kafka:9092`	Broker list
`TOPIC`	`keycloak.events`	Target topic
`EVENT_TYPES`	`LOGIN,LOGIN_ERROR,REGISTER`	Comma list; `*` = all; unknown names warned and skipped
`CLIENT_ID`	`keycloak-event-listener`	Kafka client id
`QUEUE_SIZE`	`10000`	Events waiting to be sent
`USERNAME` / `PASSWORD`	–	SASL credentials; password never logged
`SASL_MECHANISM`	`PLAIN`	`PLAIN`, `SCRAM-SHA-256`, `SCRAM-SHA-512`
`SECURITY_PROTOCOL`	`SASL_SSL` if username set, else `PLAINTEXT`	Use `SASL_PLAINTEXT` for a local broker without TLS
`SSL_TRUSTSTORE_LOCATION` / `_PASSWORD`	–	Only for brokers with a private CA

Kafka provider	Mechanism	Protocol
Confluent Cloud	PLAIN	SASL_SSL
Aiven, AWS MSK (SCRAM), Redpanda Cloud	SCRAM-SHA-512 / -256	SASL_SSL
Local broker in Docker without TLS	as configured on the broker	SASL_PLAINTEXT

5.4 Message format

```
headers: eventType=LOGIN        key: 29b96dc2-da20-436e-8a63-39861545f4ed
{"id":"…","time":1792181651000,"type":"LOGIN","realmId":"…","realmName":"greshfi",
 "clientId":"admin-cli","userId":"29b9…","sessionId":"…","ipAddress":"…","error":null,
 "details":{"auth_method":"openid-connect","username":"todimu"}}
```

> **Delivery is best-effort**: if Kafka is down, events are dropped (logged with a count on reconnect). For guaranteed delivery use the outbox pattern (write to a DB table, publish from there). Events contain usernames and IPs (personal data): set topic retention, e.g. 7 days.

5.5 Turning it on and operating it

1. Build the image (the Dockerfile copies the extension jar before `kc.sh build`).
2. Set the env vars in `.env`; restart the nodes. Look for `Kafka event listener ready: servers=… auth=… eventTypes=[…]`.
3. Realm settings → Events → Event listeners: add `kafka` **per realm** (master covers only admin-console logins).
4. Expected first-time log: `UNKNOWN_TOPIC_OR_PARTITION` (recoverable) when the topic is auto-created. Better: create it explicitly with partitions and retention, then disable auto-create.
5. Per-event debug lines: `KC_LOG_LEVEL=INFO,com.todimu.keycloak.events:DEBUG`.

Log line	Meaning
`cannot create Kafka producer (…); events are dropped, retrying in 30s`	Broker unreachable or name doesn't resolve; Keycloak unaffected
`connected producer to Kafka / N event(s) were dropped…`	Recovered
`failed to publish LOGIN event … Authentication failed`	Wrong credentials, or broker not recreated after a password change
`queue full or shutting down, dropped …`	Kafka down long enough to fill the queue

5.6 How it was verified

Inside a real Keycloak 26.7.3: factory loading and config parsing (including the typo warning), producer creation with PLAIN and SCRAM-SHA-512 (Kafka logs "Successfully logged in", password shown as `[hidden]`), `LOGIN` and `LOGIN_ERROR` reaching the publisher, login latency with Kafka down, startup with an unresolvable host and automatic recovery. On the local compose stack with a SASL-enabled broker, the listener connected and authenticated, the broker auto-created `keycloak.events` on the first event, and no publish failures were logged. The temporary Kafka broker was removed from `docker-compose.yaml` afterwards; the extension remains in the image.

6. Deployments

> Sources: section 6.1 comes from notes of earlier conversations about the AWS deployment at `auth.todimu.net` ; section 6.2 is taken directly from the `keycloak-setup` repository.

6.1 Single node on AWS EC2 ("production-style")

```
Internet ──► Nginx (container, :80/:443, TLS termination, Let's Encrypt)
               │  proxy_pass http://iam:8080      (iam = Compose service name)
               ▼
          Keycloak (custom image, start --optimized, HTTP :8080)
               │
               ▼
          MySQL (container, persistent Docker volume, :3306)
```

Aspect	What was built
Host	AWS EC2, Docker Compose
Public hostname	`auth.todimu.net`
Keycloak settings	`KC_HOSTNAME=auth.todimu.net` , `KC_HTTP_ENABLED=true` , `KC_PROXY_HEADERS=xforwarded` (not the deprecated `KC_PROXY=edge`)
TLS	Let's Encrypt + Certbot, webroot challenge; certificate works
Admin protection	Admin paths restricted at Nginx to an IP allowlist
Mode	Production mode with an optimized custom image, not `start-dev`

> **Outstanding hardening (as listed at the time):** certificate auto-renewal, secrets management, database backups, monitoring and alerting, health checks, image/version management, EC2 hardening, high availability.

6.2 Three-node cluster (keycloak-setup repository)

```
            :8080
Browser ──► nginx-lb (least_conn, max_fails=1, fail_timeout=5s)
              ├──► iam-1 :8080 ┐
              ├──► iam-2 :8080 ├── Infinispan cluster (KC_CACHE=ispn,
              └──► iam-3 :8080 ┘   discovery KC_CACHE_STACK=jdbc-ping via the DB)
                         │
                         ▼
                  MySQL 8.0.39 (iam-volume)        MailHog (SMTP 1025, UI 8025)
```

Aspect	How it works
Same image everywhere	`todimuisewon/customized-iam-customized:v1.0.0` , built by the multi-stage Dockerfile (Maven → `kc.sh build` → runtime)

Aspect	How it works
Clustering	`KC_CACHE=ispn` enables distributed Infinispan caches; `KC_CACHE_STACK=jdbc-ping` lets nodes find each other through a table in the shared database, so no multicast is needed (works in containers and cloud networks)
Sessions survive a node loss	Sessions live in the distributed cache. Test: log in, kill the node you're on, refresh; you should stay logged in
Load balancing	nginx `least_conn`; passive health: a node failing once is skipped for 5 s
Proxy headers	nginx sends `X-Forwarded-*`; Keycloak trusts them via `--proxy-headers=xforwarded`
Config	All nodes share `.env` via `env_file`; `.env` is in `.gitignore` and `.dockerignore`
Startup order	Nodes wait for MySQL's healthcheck (`condition: service_healthy`)

Operating the cluster

```
docker build -t todimuisewon/customized-iam-customized:v1.0.0 .
docker compose up -d --force-recreate iam-1 iam-2 iam-3      # recreate ALL nodes after a rebuild
docker compose stop iam-1      # graceful stop       docker compose kill iam-1   # simulate a crash
docker compose up -d iam-1     # bring it back (manual stop is not auto-restarted with unless-stopped)
docker compose logs -f iam-1 iam-2 iam-3
```

Improvements (not yet done)

- **Real health checks**: the Keycloak nodes use `test: ["CMD", "true"]` (always healthy). Use the management port: `/health/ready` on `:9000` (`KC_HEALTH_ENABLED=true` is already set). The Keycloak image has no curl; a common approach is a bash `/dev/tcp` request or an external check.
- **Active health checks / sticky sessions** at the load balancer (nginx OSS only does passive checks).
- **Secrets**: MySQL root password is in the compose file; move it to `.env` or a secrets manager.
- **TLS** in front of the cluster (as in the single-node setup) and a public `KC_HOSTNAME`.
- Part 2 of the article series targets a cluster on **AWS ECS** (in progress at the time of the notes); part 3, Kubernetes.

7. Command cheat sheet

Task	Command / place
Build image	`docker build -t todimuisewon/customized-iam-customized:v1.0.0 .`
Restart all nodes on new image	`docker compose up -d --force-recreate iam-1 iam-2 iam-3`
Remove containers no longer in compose	`docker compose up -d --remove-orphans`
Logs (live, all nodes)	`docker compose logs -f iam-1 iam-2 iam-3`
Only listener logs	`docker compose logs -f iam-1 | grep -i "kafka event listener"`
Check jar is in image	`docker compose exec iam-1 ls /opt/keycloak/providers`
Image build time	`docker image inspect <image> --format '{{.Created}}'`
Theme dev mode	`start-dev --spi-theme--cache-themes=false --spi-theme--cache-templates=false`
See CSS changes	Incognito window or DevTools → Network → Disable cache
Debug logs for your package	`KC_LOG_LEVEL=INFO,com.todimu.keycloak.events:DEBUG`
Keycloak's own event history	Realm settings → Events → Save events, then the Events tab
Read a Kafka topic (SASL)	`kafka-console-consumer.sh --bootstrap-server kafka:9092 --consumer.config client.properties --topic keycloak.events --from-beginning --property print.key=true --property print.headers=true`
Count messages	`kafka-get-offsets.sh --bootstrap-server kafka:9092 --command-config client.properties --topic keycloak.events`
Admin API: set password + verify email	`executeActionsEmail(List.of("VERIFY_EMAIL", "UPDATE_PASSWORD"))`

Further reading

- Keycloak Server Developer Guide: "Service Provider Interfaces", "Authentication SPI", "User Storage SPI" (keycloak.org/docs/latest/server_development)
- github.com/keycloak/keycloak-quickstarts, `extension/` folder
- Niko Köbler (github.com/dasniko): keycloak-extensions-demo, book *Keycloak – Identity and Access Management*
- github.com/p2-inc/keycloak-events: production-grade events extension

- keycloak.org/extensions: community extension list

Appendix A: Kafka event listener source

Final versions from `keycloak-cutomization/src/main/java/com/todimu/keycloak/events/`.

KafkaEventListenerProviderFactory.java

```
package com.todimu.keycloak.events;

import org.apache.kafka.clients.CommonClientConfigs;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.config.SaslConfigs;
import org.apache.kafka.common.config.SslConfigs;
import org.apache.kafka.common.serialization.StringSerializer;
import org.jboss.logging.Logger;
import org.keycloak.Config;
import org.keycloak.events.EventListenerProvider;
import org.keycloak.events.EventListenerProviderFactory;
import org.keycloak.events.EventType;
import org.keycloak.models.KeycloakSession;
import org.keycloak.models.KeycloakSessionFactory;

import java.util.Collections;
import java.util.EnumSet;
import java.util.Properties;
import java.util.Set;


public class KafkaEventListenerProviderFactory implements EventListenerProviderFactory {

    public static final String PROVIDER_ID = "kafka";

    private static final Logger log = Logger.getLogger(KafkaEventListenerProviderFactory.class);

    private static final String DEFAULT_EVENT_TYPES = "LOGIN,LOGIN_ERROR,REGISTER";

    private KafkaEventPublisher publisher;

    private Set<EventType> eventTypes;

    @Override
    public String getId() {
        return PROVIDER_ID;
    }

    @Override
    public void init(Config.Scope config) {
        String servers = config.get("bootstrap-servers", "kafka:9092");
        String clientId = config.get("client-id", "keycloak-event-listener");
        String topic = config.get("topic", "keycloak.events");
        int queueSize = config.getInt("queue-size", 10000);
        eventTypes = parseEventTypes(config.get("event-types", DEFAULT_EVENT_TYPES));

        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, servers);
        props.put(ProducerConfig.CLIENT_ID_CONFIG, clientId);
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        props.put(ProducerConfig.ACKS_CONFIG, "all");
        props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
        props.put(ProducerConfig.MAX_BLOCK_MS_CONFIG, 2000);
        props.put(ProducerConfig.DELIVERY_TIMEOUT_MS_CONFIG, 30000);
        props.put(ProducerConfig.REQUEST_TIMEOUT_MS_CONFIG, 10000);
        props.put(ProducerConfig.LINGER_MS_CONFIG, 5);
```

```
        String auth = configureSecurity(config, props);
        // The producer is created later, on the publisher's background thread, so an
        // unreachable Kafka can never stop Keycloak from starting.
        publisher = new KafkaEventPublisher(() -> createProducer(props), topic, queueSize);
        publisher.start();

        log.infof("Kafka event listener ready: servers=%s topic=%s auth=%s eventTypes=%s", servers,
topic, auth, eventTypes);
    }

    @Override
    public EventListenerProvider create(KeycloakSession session) {
        return new KafkaEventListenerProvider(session, publisher, eventTypes);
    }

    @Override
    public void postInit(KeycloakSessionFactory factory) {
        // nothing to do
    }

    @Override
    public void close() {
        if (publisher != null) {
            // sends what is still queued, flushes and closes the producer
            publisher.close();
        }
    }

    /**
     * Kafka loads classes by name (serializers, SASL login modules) through the
     * thread's context class loader. Under Keycloak that loader can't see this
     * jar, so point it at the loader that loaded Kafka (our jar) while creating
     * the producer.
     */
    private static KafkaProducer<String, String> createProducer(Properties props) {
        ClassLoader original = Thread.currentThread().getContextClassLoader();
        try {
            Thread.currentThread().setContextClassLoader(KafkaProducer.class.getClassLoader());
            return new KafkaProducer<>(props);
        } finally {
            Thread.currentThread().setContextClassLoader(original);
        }
    }

    static String configureSecurity(Config.Scope config, Properties props) {
        String username = config.get("username");
        String password = config.get("password");
        boolean hasCredentials = username != null && !username.isBlank();

        String protocol = config.get("security-protocol", hasCredentials ? "SASL_SSL" :
"PLAINTEXT").trim().toUpperCase();
        props.put(CommonClientConfigs.SECURITY_PROTOCOL_CONFIG, protocol);

        String truststore = config.get("ssl-truststore-location");
        if (truststore != null && !truststore.isBlank()) {
            props.put(SslConfigs.SSL_TRUSTSTORE_LOCATION_CONFIG, truststore);
            String truststorePassword = config.get("ssl-truststore-password");
            if (truststorePassword != null) {
                props.put(SslConfigs.SSL_TRUSTSTORE_PASSWORD_CONFIG, truststorePassword);
            }
        }

        if (!protocol.startsWith("SASL")) {
            return protocol;
        }
        if (!hasCredentials || password == null) {
            throw new IllegalStateException("Kafka event listener: security-protocol " + protocol
                    + " needs both 'username' and 'password' to be set");
```

```
        }

        String mechanism = config.get("sasl-mechanism", "PLAIN").trim().toUpperCase();
        String loginModule = switch (mechanism) {
            case "PLAIN" -> "org.apache.kafka.common.security.plain.PlainLoginModule";
            case "SCRAM-SHA-256", "SCRAM-SHA-512" ->
"org.apache.kafka.common.security.scram.ScramLoginModule";
            default -> throw new IllegalStateException("Kafka event listener: unsupported sasl-
mechanism '" + mechanism
                    + "' (use PLAIN, SCRAM-SHA-256 or SCRAM-SHA-512)");
        };
        props.put(SaslConfigs.SASL_MECHANISM, mechanism);
        props.put(SaslConfigs.SASL_JAAS_CONFIG, loginModule + " required"
                + " username=\"" + jaasEscape(username) + "\""
                + " password=\"" + jaasEscape(password) + "\";");

        return protocol + "/" + mechanism + " as " + username;
    }

    /** Escapes backslashes and quotes so any password works inside the JAAS string. */
    private static String jaasEscape(String value) {
        return value.replace("\\", "\\\\").replace("\"", "\\\"");
    }

    /** Turns "LOGIN, LOGIN_ERROR,REGISTER" into a set; "*" means every type; unknown names are logged
and skipped. */
    static Set<EventType> parseEventTypes(String value) {
        if (value == null || value.isBlank()) {
            return Collections.emptySet();
        }
        if (value.trim().equals("*")) {
            return Collections.unmodifiableSet(EnumSet.allOf(EventType.class));
        }
        Set<EventType> types = EnumSet.noneOf(EventType.class);
        for (String name : value.split(",")) {
            String trimmed = name.trim().toUpperCase();
            if (trimmed.isEmpty()) {
                continue;
            }
            try {
                types.add(EventType.valueOf(trimmed));
            } catch (IllegalArgumentException e) {
                log.warnf("Kafka event listener: ignoring unknown event type '%s'", trimmed);
            }
        }
        return Collections.unmodifiableSet(types);
    }
}
```

KafkaEventListenerProvider.java

```
package com.todimu.keycloak.events;

import org.jboss.logging.Logger;
import org.keycloak.events.Event;
import org.keycloak.events.EventListenerProvider;
import org.keycloak.events.EventType;
import org.keycloak.events.admin.AdminEvent;
import org.keycloak.models.AbstractKeycloakTransaction;
import org.keycloak.models.KeycloakSession;
import org.keycloak.models.RealmModel;
import org.keycloak.util.JsonSerialization;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Set;
```

```
public class KafkaEventListenerProvider implements EventListenerProvider {

    private static final Logger log = Logger.getLogger(KafkaEventListenerProvider.class);

    private final KeycloakSession session;

    private final KafkaEventPublisher publisher;

    private final Set<EventType> eventTypes;

    public KafkaEventListenerProvider(KeycloakSession session, KafkaEventPublisher publisher,
Set<EventType> eventTypes) {
        this.session = session;
        this.publisher = publisher;
        this.eventTypes = eventTypes;
    }

    @Override
    public void onEvent(Event event) {
        if (!eventTypes.contains(event.getType())) {
            return; // not one we publish
        }

        String key = event.getUserId() != null ? event.getUserId() : event.getRealmId();
        String type = event.getType().name();
        String json;
        try {
            json = JsonSerialization.writeValueAsString(toPayload(event));
        } catch (Exception e) {
            log.warnf("Kafka event listener: could not serialize %s event: %s", type, e.getMessage());
            return;
        }

        session.getTransactionManager().enlistAfterCompletion(new AbstractKeycloakTransaction() {
            @Override
            protected void commitImpl() {
                publisher.publish(key, type, json); // queued; returns immediately
            }

            @Override
            protected void rollbackImpl() {
                // transaction failed: nothing happened, nothing to publish
            }
        });
    }

    @Override
    public void onEvent(AdminEvent event, boolean includeRepresentation) {
        // Admin events are not published (yet).
    }

    @Override
    public void close() {
        // nothing to close: the publisher is shared and owned by the factory
    }

    private Map<String, Object> toPayload(Event event) {
        Map<String, Object> payload = new LinkedHashMap<>();
        payload.put("id", event.getId());
        payload.put("time", event.getTime());
        payload.put("type", event.getType().name());
        payload.put("realmId", event.getRealmId());
        payload.put("realmName", realmName(event.getRealmId()));
        payload.put("clientId", event.getClientId());
        payload.put("userId", event.getUserId());
        payload.put("sessionId", event.getSessionId());
        payload.put("ipAddress", event.getIpAddress());
```

```
        payload.put("error", event.getError());
        payload.put("details", event.getDetails());
        return payload;
    }

    private String realmName(String realmId) {
        if (realmId == null) {
            return null;
        }
        RealmModel realm = session.realms().getRealm(realmId);
        return realm != null ? realm.getName() : null;
    }
}
```

KafkaEventPublisher.java

```
package com.todimu.keycloak.events;

import org.apache.kafka.clients.producer.Producer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.jboss.logging.Logger;

import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.RejectedExecutionException;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
import java.util.function.Supplier;


public class KafkaEventPublisher implements AutoCloseable {

    private static final Logger log = Logger.getLogger(KafkaEventPublisher.class);
    private static final long RETRY_INTERVAL_MS = 30_000;

    private final Supplier<Producer<String, String>> producerFactory;
    private final String topic;
    private final ExecutorService sender;

    // only touched by the single sender thread (and close() after it has stopped)
    private Producer<String, String> producer;
    private long nextAttemptAt = 0;
    private long droppedSinceLastWarning = 0;

    public KafkaEventPublisher(Supplier<Producer<String, String>> producerFactory, String topic, int
queueSize) {
        this.producerFactory = producerFactory;
        this.topic = topic;
        this.sender = new ThreadPoolExecutor(
                1, 1, 0L, TimeUnit.MILLISECONDS,
                new ArrayBlockingQueue<>(queueSize),
                runnable -> {
                    Thread thread = new Thread(runnable, "kafka-event-publisher");
                    thread.setDaemon(true);
                    return thread;
                },
                new ThreadPoolExecutor.AbortPolicy());
    }

    /** Tries to connect right away (in the background) so problems show up in the startup logs. */
    public void start() {
        try {
            sender.execute(this::producer);
        } catch (RejectedExecutionException ignored) {
```

```
            // shutting down
        }
    }

    public void publish(String key, String eventType, String json) {
        try {
            sender.execute(() -> send(key, eventType, json));
        } catch (RejectedExecutionException e) {
            log.warnf("Kafka event listener: queue full or shutting down, dropped %s event (key=%s)",
eventType, key);
        }
    }

    private void send(String key, String eventType, String json) {
        Producer<String, String> p = producer();
        if (p == null) {
            droppedSinceLastWarning++;
            return; // Kafka not available yet; reported by producer()
        }
        try {
            ProducerRecord<String, String> record = new ProducerRecord<>(topic, key, json);
            // consumers can route/filter on this header without parsing the JSON
            record.headers().add("eventType", eventType.getBytes(StandardCharsets.UTF_8));
            p.send(record, (metadata, error) -> {
                if (error != null) {
                    log.warnf("Kafka event listener: failed to publish %s event (key=%s): %s",
eventType, key, error.getMessage());
                } else if (log.isDebugEnabled()) {
                    log.debugf("Kafka event listener: published %s to %s-%d@%d",
                            eventType, metadata.topic(), metadata.partition(), metadata.offset());
                }
            });
        } catch (Exception e) {
            log.warnf("Kafka event listener: could not send %s event (key=%s): %s", eventType, key,
e.getMessage());
        }
    }

    /** Returns the producer, creating it if needed; null while Kafka can't be reached (retried every
30s). */
    private Producer<String, String> producer() {
        if (producer != null) {
            return producer;
        }
        long now = System.currentTimeMillis();
        if (now < nextAttemptAt) {
            return null;
        }
        try {
            producer = producerFactory.get();
            log.info("Kafka event listener: connected producer to Kafka");
            if (droppedSinceLastWarning > 0) {
                log.warnf("Kafka event listener: %d event(s) were dropped while Kafka was unavailable",
droppedSinceLastWarning);
                droppedSinceLastWarning = 0;
            }
            return producer;
        } catch (Exception e) {
            nextAttemptAt = now + RETRY_INTERVAL_MS;
            Throwable cause = e.getCause() != null ? e.getCause() : e;
            log.warnf("Kafka event listener: cannot create Kafka producer (%s); events are dropped,
retrying in %ds",
                    cause.getMessage(), RETRY_INTERVAL_MS / 1000);
            return null;
        }
    }

    @Override
```

```
    public void close() {
        sender.shutdown();
        try {
            if (!sender.awaitTermination(5, TimeUnit.SECONDS)) {
                sender.shutdownNow();
            }
        } catch (InterruptedException e) {
            sender.shutdownNow();
            Thread.currentThread().interrupt();
        }
        if (producer != null) {
            producer.close(Duration.ofSeconds(5));
        }
    }
}
```

src/main/resources/META-INF/services/org.keycloak.events.EventListenerProviderFactory

```
com.todimu.keycloak.events.KafkaEventListenerProviderFactory
```

keycloak-cutomization/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>com.todimu.keycloak</groupId>
        <artifactId>keycloak-setup</artifactId>
        <version>1.0.0-SNAPSHOT</version>
    </parent>

    <artifactId>keycloak-cutomization</artifactId>
    <name>keycloak-extensions</name>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <keycloak.version>26.7.3</keycloak.version>
        <kafka.version>3.9.1</kafka.version>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.keycloak</groupId>
            <artifactId>keycloak-server-spi</artifactId>
            <version>${keycloak.version}</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>org.keycloak</groupId>
            <artifactId>keycloak-server-spi-private</artifactId>
            <version>${keycloak.version}</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>org.keycloak</groupId>
            <artifactId>keycloak-core</artifactId>
            <version>${keycloak.version}</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
```

```
            <groupId>org.jboss.logging</groupId>
            <artifactId>jboss-logging</artifactId>
            <version>3.6.1.Final</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>org.apache.kafka</groupId>
            <artifactId>kafka-clients</artifactId>
            <version>${kafka.version}</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-shade-plugin</artifactId>
                <version>3.6.0</version>
                <executions>
                    <execution>
                        <phase>package</phase>
                        <goals><goal>shade</goal></goals>
                        <configuration>
                            <createDependencyReducedPom>false</createDependencyReducedPom>
                            <artifactSet>
                                <excludes>
                                    <!-- Keycloak already ships slf4j; two copies conflict -->
                                    <exclude>org.slf4j:*</exclude>
                                </excludes>
                            </artifactSet>
                            <transformers>
                                <transformer
implementation="org.apache.maven.plugins.shade.resource.ServicesResourceTransformer"/>
                            </transformers>
                        </configuration>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>
</project>
```

Appendix B: Theme source

theme/greshfi/login/resources/css/greshfi.css

```
/* ========================================================
   Greshfi login — simple design (CSS only, on top of keycloak.v2)
   Change the brand colour in one place: --greshfi-primary
   ======================================================== */

:root {
  --greshfi-primary: #4f46e5;
  --greshfi-primary-hover: #4338ca;
  --greshfi-bg: #f5f6fa;
  --greshfi-card: #ffffff;
  --greshfi-text: #111827;
  --greshfi-muted: #6b7280;
  --greshfi-border: #e5e7eb;
  --greshfi-radius: 14px;

  /* feed the brand colour into PatternFly */
  --pf-v5-global--primary-color--100: var(--greshfi-primary);
  --pf-v5-global--primary-color--200: var(--greshfi-primary-hover);
  --pf-v5-global--link--Color: var(--greshfi-primary);
  --pf-v5-global--link--Color--hover: var(--greshfi-primary-hover);
}

/* dark mode (keycloak.v2 adds this class when the OS is in dark mode) */
.pf-v5-theme-dark {
  --greshfi-primary: #818cf8;
  --greshfi-primary-hover: #6366f1;
  --greshfi-bg: #0f1115;
  --greshfi-card: #171a21;
  --greshfi-text: #f3f4f6;
  --greshfi-muted: #9ca3af;
  --greshfi-border: #2a2f3a;
}

/* page background: plain colour instead of the Keycloak artwork */
.login-pf body {
  background: var(--greshfi-bg);
}

/* realm name above the card: normal case, dark text */
#kc-header-wrapper {
  color: var(--greshfi-text) !important;
  font-size: 1.5rem;
  font-weight: 700;
  letter-spacing: -0.01em;
  text-transform: none;
}

/* narrower card */
.pf-v5-c-login__container {
  grid-template-columns: minmax(0, 26rem);
}

/* the card */
.pf-v5-c-login__main {
  background: var(--greshfi-card);
  border: 1px solid var(--greshfi-border);
  border-radius: var(--greshfi-radius);
  box-shadow: 0 1px 2px rgba(16, 24, 40, .04), 0 12px 32px rgba(16, 24, 40, .08);
  overflow: hidden;
}
```

```
/* remove the blue bar on top of the card */
.pf-v5-c-login__main-header {
  border-top: none;
  padding-top: 2rem;
}

/* "Sign in to your account" */
#kc-page-title {
  color: var(--greshfi-text);
  font-size: 1.25rem;
  font-weight: 600;
}

/* inputs: one clean rounded box, brand-coloured focus ring
   (PatternFly draws borders with ::before/::after — turn those off
   and draw our own) */
.pf-v5-c-form-control::before,
.pf-v5-c-form-control::after,
.pf-v5-c-input-group .pf-v5-c-button.pf-m-control::before,
.pf-v5-c-input-group .pf-v5-c-button.pf-m-control::after {
  border: none !important;
}
.pf-v5-c-form-control,
.pf-v5-c-input-group {
  background: var(--greshfi-card);
  border: 1px solid var(--greshfi-border);
  border-radius: 10px;
  overflow: hidden;
  transition: border-color .15s, box-shadow .15s;
}
.pf-v5-c-input-group .pf-v5-c-form-control,
.pf-v5-c-input-group .pf-v5-c-button.pf-m-control {
  border: none;
  border-radius: 0;
  background: transparent;
}
.pf-v5-c-form-control > input {
  padding-block: .65rem;
  outline: none;
}
.pf-v5-c-form-control:not(.pf-v5-c-input-group .pf-v5-c-form-control):focus-within,
.pf-v5-c-input-group:focus-within {
  border-color: var(--greshfi-primary);
  box-shadow: 0 0 0 3px color-mix(in srgb, var(--greshfi-primary) 22%, transparent);
}
/* keep error states visible */
.pf-v5-c-form-control.pf-m-error,
.pf-v5-c-form-control:has(input[aria-invalid="true"]),
.pf-v5-c-input-group:has(input[aria-invalid="true"]) {
  border-color: var(--pf-v5-global--danger-color--100) !important;
  box-shadow: 0 0 0 3px color-mix(in srgb, var(--pf-v5-global--danger-color--100) 18%, transparent)
!important;
}

/* main button: full width, rounded */
.pf-v5-c-button.pf-m-primary {
  --pf-v5-c-button--m-primary--BackgroundColor: var(--greshfi-primary);
  --pf-v5-c-button--m-primary--hover--BackgroundColor: var(--greshfi-primary-hover);
  --pf-v5-c-button--m-primary--focus--BackgroundColor: var(--greshfi-primary-hover);
  --pf-v5-c-button--BorderRadius: 10px;
  border-radius: 10px;
  font-weight: 600;
}
#kc-login {
  width: 100%;
  padding-block: .7rem;
}
```

```
/* small print under the card ("New user? Register") */
.pf-v5-c-login__main-footer-band {
  color: var(--greshfi-muted);
  border-top-color: var(--greshfi-border);
}

/* logo: icon + wordmark in one image (Option B).
   Needs the realm's HTML Display name set to:
   <div class="kc-logo-text"><span>Greshfi</span></div> */
div.kc-logo-text {
  background-image: url("../img/logo-dark-text.svg");
  width: 200px;
  height: 58px;
}
.pf-v5-theme-dark div.kc-logo-text {
  background-image: url("../img/logo-light-text.svg");
}
```

theme/greshfi/email/html/template.ftl

```
<#--
  Greshfi email wrapper: every HTML email is rendered inside this layout.
  The text of each email comes from its "...BodyHtml" key in messages/messages_en.properties.
  The greeting ("Hi {first name},") comes from the emailGreeting key and is skipped if the user has no
first name.

  Email apps ignore external CSS, so styles are inline (plus a small <style> block
  for the links and paragraphs that come from the message text).

  Logo: set logoUrl in theme.properties to a PUBLIC https URL of a PNG
  (Gmail/Outlook don't show SVG). Without it, a text wordmark is shown.
-->
<#macro emailLayout>
<!DOCTYPE html>
<html lang="${locale.language}" dir="${(ltr)?then('ltr','rtl')}">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="color-scheme" content="light">
  <meta name="supported-color-schemes" content="light">
  <title>${realmName!""}</title>
  <style>
    .gf-body p { margin: 0 0 16px; }
    .gf-body p:last-child { margin-bottom: 0; }
    .gf-body a { color: #15803d; font-weight: 600; text-decoration: underline; }
    .gf-body b, .gf-body strong { color: #0f1d1a; }
    @media (max-width: 600px) {
      .gf-card { padding: 28px 22px !important; }
    }
  </style>
</head>
<body style="margin:0; padding:0; background:#f5f6fa; -webkit-text-size-adjust:100%;">
  <table role="presentation" width="100%" cellpadding="0" cellspacing="0" border="0"
style="background:#f5f6fa;">
    <tr>
      <td align="center" style="padding:32px 16px;">
        <table role="presentation" width="100%" cellpadding="0" cellspacing="0" border="0" style="max-
width:560px;">

          <!-- header -->
          <tr>
            <td align="center" style="background:#0f1d1a; border-radius:14px 14px 0 0; padding:26px
24px;">
              <#if properties.logoUrl?has_content>
                <img src="${properties.logoUrl}" alt="${realmName!"Gresh"}" height="40"
style="display:block; height:40px; width:auto; border:0;">
```

```
              <#else>
                <span style="font-family:Arial,Helvetica,sans-serif; font-size:26px; font-weight:700;
letter-spacing:-0.5px; color:#ffffff;">Gresh<span style="color:#66DD8C;">.</span></span>
              </#if>
            </td>
          </tr>

          <!-- green accent line -->
          <tr><td style="background:#66DD8C; height:4px; line-height:4px; font-size:0;"> </td>
</tr>

          <!-- body: the message text goes here -->
          <tr>
            <td class="gf-card gf-body" style="background:#ffffff; border:1px solid #e5e7eb; border-
top:none; border-radius:0 0 14px 14px; padding:36px 40px; font-family:Arial,Helvetica,sans-serif; font-
size:15px; line-height:1.6; color:#374151;">
              <#if user?? && user.firstName?has_content>
                <p style="margin:0 0 16px;">${msg("emailGreeting", user.firstName?cap_first)}</p>
              </#if>
              <#nested>
            </td>
          </tr>

          <!-- footer -->
          <tr>
            <td align="center" style="padding:22px 16px 0; font-family:Arial,Helvetica,sans-serif;
font-size:12px; line-height:1.6; color:#9ca3af;">
              This email was sent by ${realmName!"Gresh"}. If you didn't expect it, you can safely
ignore it.
            </td>
          </tr>

        </table>
      </td>
    </tr>
  </table>
</body>
</html>
</#macro>
```

Appendix C: Build and cluster files

Dockerfile

```
FROM maven:3.9.6-eclipse-temurin-21 AS MAVEN_BUILD
COPY ./ ./
RUN mvn clean package

FROM quay.io/keycloak/keycloak:26.7.3 AS builder
ENV KC_HEALTH_ENABLED=true
ENV KC_METRICS_ENABLED=true
ENV KC_FEATURES=token-exchange,admin-fine-grained-authz
ENV QUARKUS_TRANSACTION_MANAGER_ENABLE_RECOVERY=true
ENV KC_DB=mysql
WORKDIR /opt/keycloak
COPY --from=MAVEN_BUILD ./keycloak-theme/target/*.jar /opt/keycloak/providers
COPY --from=MAVEN_BUILD ./keycloak-cutomization/target/keycloak-cutomization-*.jar
/opt/keycloak/providers/
RUN /opt/keycloak/bin/kc.sh build

FROM quay.io/keycloak/keycloak:26.7.3
COPY --from=builder /opt/keycloak/ /opt/keycloak/
EXPOSE 8080
EXPOSE 9000
EXPOSE 8443
ENTRYPOINT ["/opt/keycloak/bin/kc.sh", "start", "--optimized", "--http-enabled=true", "--proxy-
headers=xforwarded"]
```

nginx-lb/default.conf

```
upstream keycloak_nodes {
    least_conn;
    server iam-1:8080 max_fails=1 fail_timeout=5s;
    server iam-2:8080 max_fails=1 fail_timeout=5s;
    server iam-3:8080 max_fails=1 fail_timeout=5s;
}

server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://keycloak_nodes;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Host $http_host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Port $server_port;
        proxy_set_header X-Real-IP $remote_addr;

        proxy_connect_timeout 3s;
        proxy_read_timeout 90s;
    }
}
```

docker-compose.yaml (excerpt; iam-2/iam-3 identical to iam-1)

```
  nginx-lb:
    image: nginx:1.27-alpine
    ports:
      - "8080:80"
```

```
    volumes:
      - ./nginx-lb/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on: [iam-1, iam-2, iam-3]
    networks: [iam-network]
    restart: unless-stopped

  iam-1:                      # iam-2 and iam-3 are identical
    image: todimuisewon/customized-iam-customized:v1.0.0
    ports:
      - "8080"
    restart: unless-stopped
    env_file:
      - .env
    networks: [iam-network]
    depends_on:
      database:
        condition: service_healthy
    healthcheck:
      test: [ "CMD", "true" ]  # placeholder, see "Improvements"
      interval: 10s
      timeout: 10s
      retries: 20
      start_period: 60s

  database:
    image: mysql:8.0.39
    environment:
      MYSQL_DATABASE: keycloak
    volumes:
      - iam-volume:/var/lib/mysql
    networks: [iam-network]
    healthcheck:
      test: [ "CMD", "mysqladmin", "ping", "-h", "localhost" ]
```

.env (keys only; values are secrets)

```
KC_BOOTSTRAP_ADMIN_USERNAME / KC_BOOTSTRAP_ADMIN_PASSWORD
KC_DB_USERNAME / KC_DB_PASSWORD / KC_DB_URL
KC_HOSTNAME
KC_HTTPS_CERTIFICATE_FILE_LOCAL / KC_HTTPS_CERTIFICATE_KEY_FILE_LOCAL
KC_CACHE=ispn
KC_CACHE_STACK=jdbc-ping
KC_SPI_EVENTS_LISTENER__KAFKA__BOOTSTRAP_SERVERS / TOPIC / EVENT_TYPES
KC_SPI_EVENTS_LISTENER__KAFKA__SECURITY_PROTOCOL / SASL_MECHANISM / USERNAME / PASSWORD
KAFKA_ADMIN_PASSWORD                (only used by the removed test broker)
```